APB Protocol Slave Module (SRAM) Verification

Interface

```
`timescale 1ns/1ps

interface apb_if #(parameter DATA_BUS_WIDTH=32, ADDR_BUS_WIDTH=32
      )(
      input logic  pclk
      );

      logic presetn;
      logic psel;
      logic penable;
      logic pwrite;



      logic [DATA_BUS_WIDTH-1:0] pwdata;
      logic [ADDR_BUS_WIDTH-1:0] paddr;
      logic [DATA_BUS_WIDTH-1:0] prdata;

      logic pready;
      logic pslverr;

endinterface
```

Transaction

```
`timescale 1ns/1ps

class transaction #(parameter DATA_BUS_WIDTH=32, ADDR_BUS_WIDTH=32);
    rand bit pwrite;
    rand logic [ADDR_BUS_WIDTH-1:0] paddr;
        rand logic [DATA_BUS_WIDTH-1:0] pwdata;
    bit psel;
    bit penable;


    logic [DATA_BUS_WIDTH-1:0] prdata;
        bit pready;
        bit pslverr;
```

```
        bit [DATA_BUS_WIDTH-1:0] exp_prdata;
        bit exp_pready;
        bit exp_pslverr;


    function void display(string tag);
        $display("[%0t] [%0s] psel=%0b penable=%0b pwrite=%0b paddr=%0h
pwdata=%0h | prdata=%0h pready=%0b pslverr=%0b", $time, tag,
        psel, penable, pwrite, paddr, pwdata,
        prdata, pready, pslverr
        );

    endfunction
endclass
```

Driver

```
`timescale 1ns/1ps
class driver #(
        parameter DATA_BUS_WIDTH=32, ADDR_BUS_WIDTH=32
        );
        mailbox gen2drv;
        virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif;
        transaction #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) tr;

        function new(mailbox gen2drv, virtual apb_if #(DATA_BUS_WIDTH,
ADDR_BUS_WIDTH) vif);
                this.gen2drv = gen2drv;
                this.vif     = vif;
                $display("[%0t] DRIVER CONSTRUCTED", $time);
        endfunction

        task run;
        forever begin
                        gen2drv.get(tr);

                        // SETUP phase:
                        @(posedge vif.pclk);
                        vif.psel    <= 1'b1;
                        vif.penable <= 1'b0;
                        vif.pwrite  <= tr.pwrite;
                        vif.paddr   <= tr.paddr;
```

```
                        vif.pwdata  <= tr.pwdata;

                        // ACCESS phase
                        @(posedge vif.pclk);
                        vif.penable <= 1'b1;

            wait (vif.pready);
            @(negedge vif.pclk);

                        tr.pready  = vif.pready;
            tr.display("DRIVER");


                        @(posedge vif.pclk);
                        vif.psel    <= 1'b0;
                        vif.penable <= 1'b0;
            vif.pwrite  <= 1'b0;
                end
        endtask
endclass
```

Monitor

```
`timescale 1ns/1ps
class monitor #(parameter DATA_BUS_WIDTH=32, ADDR_BUS_WIDTH=32
        );
        mailbox mon2sb;
        virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif;

        function new(mailbox mon2sb,  virtual apb_if #(DATA_BUS_WIDTH,
ADDR_BUS_WIDTH) vif);
                this.mon2sb  = mon2sb;

                this.vif     = vif;
                $display("[%0t] MONITOR CONSTRUCTED", $time);
        endfunction
task run();
    @(posedge vif.pclk iff vif.presetn === 1'b1); //wait until reset is
deasserted

    forever begin
        transaction #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) mon_tr;
            mon_tr = new();
```

```
            // Wait for SETUP phase
               do @(posedge vif.pclk); while (!(vif.psel === 1'b1 &&
vif.penable === 1'b0));
        @(negedge vif.pclk);
            mon_tr.psel   = vif.psel;
            mon_tr.pwrite = vif.pwrite;
            mon_tr.paddr  = vif.paddr;
            mon_tr.pwdata = vif.pwdata;

            // Wait for ACCESS phase
            do @(posedge vif.pclk); while (!(vif.psel === 1'b1 &&
vif.penable === 1'b1));
        @(negedge vif.pclk);
            mon_tr.penable = vif.penable;

            // Wait for pready
            while (!vif.pready)
                @(negedge vif.pclk);

            mon_tr.prdata  = vif.prdata;
            mon_tr.pready  = vif.pready;
            mon_tr.pslverr = vif.pslverr;
            mon_tr.display("MONITOR");
            mon2sb.put(mon_tr);
        end
endtask
endclass
```

Generator

```
class generator #(
    parameter DATA_BUS_WIDTH = 32,
    parameter ADDR_BUS_WIDTH = 32
);

    mailbox gen2drv;
    int count;
    virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif;


    function new(mailbox gen2drv, virtual apb_if #(DATA_BUS_WIDTH,
```

```
ADDR_BUS_WIDTH) vif);
        this.gen2drv = gen2drv;
    this.vif=vif;
    endfunction

task run(
    input bit use_test_addr = 0,
    input bit [ADDR_BUS_WIDTH-1:0] test_addr[] = {},
           input bit test_pwrite[] = {}
    );

    for (int i = 0; i < count; i++) begin
        transaction #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) tr;
        tr = new();
        if (!tr.randomize() with {
            paddr inside {[0:128]};
            })
            $error("RANDOMIZATION FAILED");

                   if (use_test_addr) begin
                tr.paddr  = test_addr[i];
                tr.pwrite = test_pwrite[i];
            end
            tr.display("GENERATOR");
            gen2drv.put(tr);
        end
endtask

task write();

        repeat(count) begin
            transaction #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) tr;
            tr=new();

            if(!tr.randomize() with {
                paddr inside {[0:63]};
                pwrite==1;
                })
                $error("RANDOMIZATION FAILED");

        tr.display("GENERATOR");
        gen2drv.put(tr);
    end
```

```
    endtask

    task read();

        repeat(count) begin
            transaction #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) tr;
            tr=new();

            if(!tr.randomize() with {
                paddr inside {[0:63]};
                pwrite==0;
                })
                $error("RANDOMIZATION FAILED");

        tr.display("GENERATOR");
        gen2drv.put(tr);
    end

    endtask
endclass
```

Agent

```
`timescale 1ns/1ps
class agent #(parameter DATA_BUS_WIDTH=32, ADDR_BUS_WIDTH=32);

        driver drv;
        monitor mon;
        generator gen;

        mailbox gen2drv;
        virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif;

        function new(mailbox mon2sb, virtual apb_if #(DATA_BUS_WIDTH,
ADDR_BUS_WIDTH) vif);
                this.vif=vif;
        gen2drv=new();
                drv=new(gen2drv, vif);
                mon=new(mon2sb, vif);
                gen=new(gen2drv, vif );
        endfunction
```

```
    task run();
        fork
            gen.run();
            drv.run();
            mon.run();
        join_none
    endtask

endclass
```

Agent Package

```
package apb_agent_pkg;
    `include "../top/transaction.sv"
    `include "../agent/driver.sv"
    `include "../agent/monitor.sv"
    `include "../seq-lib/generator.sv"
    `include "../agent/agent.sv"
endpackage
```

Environment Package

```
package apb_env_pkg;
    import apb_agent_pkg :: * ;
       `include "../env/scoreboard.sv"
       `include "../env/env.sv"
endpackage
```

Environment

```
`timescale 1ns/1ps

class env #(parameter DATA_BUS_WIDTH=32, ADDR_BUS_WIDTH=32);
        agent agt;
        scoreboard sb;

        mailbox mon2sb;

        function new(virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif);
```

```
                mon2sb=new();
    $display("[%0t] ENV: before agent", $time);
                agt=new(mon2sb, vif);
    $display("[%0t] ENV: after agent", $time);
            $display("[%0t] ENV: before scoreboard", $time);
           sb=new(mon2sb);
        $display("[%0t] ENV: after scoreboard", $time);
        endfunction
endclass
```

Scoreboard

```
class scoreboard #(
    parameter DATA_BUS_WIDTH = 32,
    parameter ADDR_BUS_WIDTH = 32,
    parameter MEMSIZE        = 64,
    parameter MEM_BLOCK_SIZE = 8,
    parameter RESET_VAL      = 0
);

    mailbox mon2sb;

    bit [MEM_BLOCK_SIZE-1:0] expected_mem [bit [ADDR_BUS_WIDTH-1:0]];



    int compare_count = 0;
    int read_pass = 0;
    int read_fail = 0;
    int error_pass= 0;
    int error_fail= 0;
    int ready_pass= 0;
    int ready_fail= 0;

    localparam int BLOCKS_PER_WORD = DATA_BUS_WIDTH / MEM_BLOCK_SIZE;
    localparam int MEM_BYTES       = MEMSIZE * BLOCKS_PER_WORD;



    function new(mailbox mon2sb);

        this.mon2sb = mon2sb;
```

```
    for (int i = 0; i < MEM_BYTES; i++) begin
            expected_mem[i] = RESET_VAL;
        end
        $display("Scoreboard Constructed");

    endfunction


    task run();

        forever begin

            transaction #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) tr;
            bit exp_pready;
            bit exp_pslverr;
            bit [DATA_BUS_WIDTH-1:0] expected_data;

            mon2sb.get(tr);


            // Only process ACCESS phase
            if (tr.psel && tr.penable) begin
            exp_pready  = 1'b1;
            exp_pslverr = 1'b0;
            expected_data = '0;

        //write

        if (tr.pwrite) begin
            if (tr.paddr <= (MEM_BYTES - BLOCKS_PER_WORD)) begin
                for (int i = 0; i < BLOCKS_PER_WORD; i++) begin
                    expected_mem[tr.paddr + i] =
                        tr.pwdata[(i * MEM_BLOCK_SIZE)+: MEM_BLOCK_SIZE];
                end
                exp_pslverr = 1'b0;
                $display("======================== DATA WRITE
============================");
            end
            else begin
                exp_pslverr = 1'b1;
            end
        end
```

```
                // READ

                else begin
            if (tr.paddr <= (MEM_BYTES - BLOCKS_PER_WORD)) begin
                expected_data = '0;
                for (int i = 0; i < BLOCKS_PER_WORD; i++) begin
                    if (expected_mem.exists(tr.paddr + i)) begin
                        expected_data[(i * MEM_BLOCK_SIZE) +:
MEM_BLOCK_SIZE] =
                            expected_mem[tr.paddr + i];
                    end
                    else begin
                        expected_data[(i * MEM_BLOCK_SIZE) +:
MEM_BLOCK_SIZE] = RESET_VAL;
                    end
                end
                exp_pslverr = 1'b0;
                $display("====================== DATA READ
============================");
            end

            else begin
                expected_data = '0;
                exp_pslverr = 1'b1;
            end

            if (tr.prdata == expected_data) begin
                read_pass++;
                $display("[%0t] READ DATA MATCH | ADDR=%0h | EXPECTED=%0h |
ACTUAL=%0h", $time, tr.paddr, expected_data, tr.prdata);
            end

            else begin
                read_fail++;
                $display("[%0t] READ DATA FAIL | ADDR=%0h | EXPECTED=%0h |
ACTUAL=%0h", $time, tr.paddr, expected_data, tr.prdata);
            end
        end

                // Compare PREADY

                if (tr.pready != exp_pready) begin
```

```
            ready_fail++;
            $display("[%0t] PREADY FAIL | EXP=%0b ACT=%0b", $time,
exp_pready, tr.pready);
        end

                // Compare PSLVERR

                if (tr.pslverr != exp_pslverr) begin
                    error_fail++;
                    $display("[%0t] PSLVERR FAIL | EXP=%0b ACT=%0b", $time,
exp_pslverr, tr.pslverr);
                end
                compare_count++;
        $display("[%0t] [SCOREBOARD] COUNT = %0d", $time, compare_count);
            end
    end
    endtask




    task report();

        $display("");
        $display("==============================");
        $display("       SCOREBOARD REPORT");
        $display("==============================");
        $display("READ PASS  = %0d", read_pass);
        $display("READ FAIL  = %0d", read_fail);
        $display("PSLVERR FAIL  = %0d", error_fail);
        $display("READY FAIL = %0d", ready_fail);
        $display("==============================");

    endtask

endclass
```

Test-lib Package

```
package test_lib_pkg;
        `include "../test-lib/base_test.sv"
```

```
        `include "../test-lib/read_write.sv"
        `include "../test-lib/deterministic.sv"
        `include "../test-lib/reset_test.sv"
        `include "../test-lib/mid_reset.sv"
        `include "../test-lib/overwrite.sv"
        `include "../test-lib/boundary_test.sv"
endpackage
```

Base Test

```
import apb_env_pkg::*;

class base_test #(
    parameter DATA_BUS_WIDTH = 32,
    parameter ADDR_BUS_WIDTH = 32
);

    env env_o;
    int total_transactions;
    virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif;

    function new(virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif);
        this.vif=vif;
        $display("[%0t] INSIDE THE BASE TEST CONSTRUCTOR", $time);
        env_o = new(vif);
    endfunction

    virtual task apply_reset(int cycles = 2);
        vif.presetn = 1'b0;
        repeat(cycles) @(posedge vif.pclk);
        vif.presetn = 1'b1;
    endtask

    virtual task run();
    apply_reset();
    env_o.agt.gen.count = total_transactions;
        env_o.agt.gen.run();
    endtask
endclass
```

Fixed Test

```
class fixed_test extends base_test #(
    DATA_BUS_WIDTH,
    ADDR_BUS_WIDTH
);


    virtual task apply_reset(int cycles = 2);
        vif.presetn = 1'b0;
        $display("[%0t] RESET ASSERTED", $time);
        repeat(cycles) @(posedge vif.pclk);
        vif.presetn = 1'b1;
        repeat(2) @(posedge vif.pclk);
        $display("[%0t] RESET DEASSERTED", $time);
    endtask


    function new(
        virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif
    );
        super.new(vif);
    endfunction

    bit [ADDR_BUS_WIDTH-1:0] test_addr[];
    bit test_pwrite[];


    virtual task run();

        apply_reset();

        test_addr = new[total_transactions];
    test_addr = '{16, 8, 17, 16, 8, 17, 64, 68, 20, 64, 68, 32, 32, 128,
253, 254, 253, 255, 254, 255};
    test_pwrite = '{1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 0, 1, 1, 1, 0,
0, 0 };

        $display("[%0t] TOTAL TRANSACTIONS = %0d",
                 $time, total_transactions);

        // RUN TEST
```

```
        env_o.agt.gen.count = test_addr.size();
        env_o.agt.gen.run(1, test_addr, test_pwrite);
    repeat(4 * 60) @(posedge vif.pclk);
    env_o.sb.report();
        $display("[%0t] END OF BOUNDARY ADDR TEST", $time);
        $finish;

    endtask

endclass
```

Read and Write from the same Address Test

```
class read_write extends base_test #(
    DATA_BUS_WIDTH,
    ADDR_BUS_WIDTH
);

    virtual task apply_reset(int cycles = 2);
        vif.presetn = 1'b0;
        $display("[%0t] RESET ASSERTED", $time);
        repeat(cycles) @(posedge vif.pclk);
        vif.presetn = 1'b1;
        repeat(2) @(posedge vif.pclk);
        $display("[%0t] RESET DEASSERTED", $time);
    endtask

    function new(
        virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif
    );
        super.new(vif);
    endfunction

    bit [ADDR_BUS_WIDTH-1:0] test_addr[];
    bit test_pwrite[];

    virtual task run();

        apply_reset();
        test_addr   = new[total_transactions];
        test_pwrite = new[total_transactions];
```

```
        for (int i = 0; i < total_transactions/2; i++) begin

            // WRITE
            test_addr[i]   = i*4;
            test_pwrite[i] = 1;

            // READ
            test_addr[i + total_transactions/2]   = i*4;
            test_pwrite[i + total_transactions/2] = 0;

        end

        $display("[%0t] TOTAL TRANSACTIONS = %0d",
                 $time, total_transactions);

        env_o.agt.gen.count = total_transactions;

        env_o.agt.gen.run(
            1,
            test_addr,
            test_pwrite
        );

    endtask

endclass
```

Reset Test

```
class reset_test extends base_test #(
    DATA_BUS_WIDTH,
    ADDR_BUS_WIDTH
);

    function new(
        virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif
    );
        super.new(vif);
    endfunction


    // Apply reset for two cycles
    virtual task apply_reset(int cycles = 2);
        vif.presetn = 1'b0;
```

```
        $display("[%0t] RESET ASSERTED", $time);
        repeat(cycles) @(posedge vif.pclk);
        vif.presetn = 1'b1;
        repeat(2) @(posedge vif.pclk);
        $display("[%0t] RESET DEASSERTED", $time);
    endtask

    //after applying reset perform reads to check whether the memory has
    //cleared out or not

    virtual task run();
        bit [ADDR_BUS_WIDTH-1:0] addresses[];
    bit test_pwrite[];
        env_o.agt.gen.count = 0;
        apply_reset();
        addresses = new[64];

          for (int i = 0; i < 64; i++) begin
            addresses[i] = i * 4;
        test_pwrite[i] = 1'b0;
        end

          env_o.agt.gen.count = 64;
    env_o.agt.gen.run(1, addresses, test_pwrite);
        $display("[%0t] END OF RESET TEST", $time);

    endtask
endclass
```

Mid Operation Reset Test

```
class mid_reset_test extends base_test #(
    DATA_BUS_WIDTH,
    ADDR_BUS_WIDTH
);

    function new(
        virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif
    );
        super.new(vif);
    endfunction
    // Apply reset for two cycles
```

```
    virtual task apply_reset(int cycles = 2);
        vif.presetn = 1'b0;
        $display("[%0t] RESET ASSERTED", $time);
        repeat(cycles) @(posedge vif.pclk);
        vif.presetn = 1'b1;
        repeat(2) @(posedge vif.pclk);
        $display("[%0t] RESET DEASSERTED", $time);
    endtask

virtual task run();

    bit [ADDR_BUS_WIDTH-1:0] addresses[];
    bit test_pwrite[];

    // =========================
    // POWER-ON RESET
    // =========================
    apply_reset();


    // =========================
    // WRITE BEFORE RESET
    // =========================
    addresses   = new[1];
    test_pwrite = new[1];

    addresses[0]   = 4;
    test_pwrite[0] = 1'b1;

    env_o.agt.gen.count = 1;
    env_o.agt.gen.run(1, addresses, test_pwrite);

    wait (env_o.sb.compare_count >= 1);

    $display("[%0t] WRITE BEFORE RESET COMPLETED", $time);


    // MID TRANSACTION RESET
    vif.presetn = 1'b0;
    $display("[%0t] RESET ASSERTED", $time);

    repeat(2) @(posedge vif.pclk);
```

```
    if (vif.pready !== 0)
        $display("MID RESET TEST FAIL: PREADY");

    if (vif.pslverr !== 0)
        $display("MID RESET TEST FAIL: PSLVERR");
    else
        $display("MID RESET TEST PASSED. PREADY=%0b, PSLVERR=%0b",
                 vif.pready, vif.pslverr);


    // DEASSERT RESET
    vif.presetn = 1'b1;
    repeat(2) @(posedge vif.pclk);

    $display("[%0t] RESET DEASSERTED", $time);

    addresses[0]   = 4;
    test_pwrite[0] = 1'b0;

    env_o.agt.gen.count = 1;
    env_o.agt.gen.run(1, addresses, test_pwrite);

    wait (env_o.sb.compare_count >= 2);

    $display("[%0t] END OF MID RESET TEST", $time);
    $finish;

endtask

endclass
```

Boundary Test

```
class boundary_test extends base_test #(
    DATA_BUS_WIDTH,
    ADDR_BUS_WIDTH
);
     virtual task apply_reset(int cycles = 2);
        vif.presetn = 1'b0;
        $display("[%0t] RESET ASSERTED", $time);
        repeat(cycles) @(posedge vif.pclk);
        vif.presetn = 1'b1;
```

```
        repeat(2) @(posedge vif.pclk);
        $display("[%0t] RESET DEASSERTED", $time);
    endtask

    function new(
        virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif
    );
        super.new(vif);
    endfunction
    bit [ADDR_BUS_WIDTH-1:0] test_addr[];
    bit test_pwrite[];
    virtual task run();

        apply_reset();
        test_addr   = new[4];
        test_pwrite = new[4];
        test_addr   = '{252, 252, 253, 253};
        test_pwrite = '{1, 0, 1, 0};

        env_o.agt.gen.count = 4;
        env_o.agt.gen.run(1, test_addr, test_pwrite);
    repeat(4 * 60) @(posedge vif.pclk);
    env_o.sb.report();

        $display("[%0t] END OF BOUNDARY ADDR TEST", $time);
        $finish;
    endtask
endclass
```

Overwrite Test

```
class overwrite_test extends base_test #(
    DATA_BUS_WIDTH,
    ADDR_BUS_WIDTH
);
     virtual task apply_reset(int cycles = 2);
        vif.presetn = 1'b0;
        $display("[%0t] RESET ASSERTED", $time);
        repeat(cycles) @(posedge vif.pclk);
        vif.presetn = 1'b1;
        repeat(2) @(posedge vif.pclk);
```

```
        $display("[%0t] RESET DEASSERTED", $time);
    endtask

    function new(
        virtual apb_if #(DATA_BUS_WIDTH, ADDR_BUS_WIDTH) vif
    );
        super.new(vif);
    endfunction
    bit [ADDR_BUS_WIDTH-1:0] test_addr[];
    bit test_pwrite[];
    virtual task run();
        apply_reset();
    // Write same address 3 times with different data, then read once
        test_addr   = new[4];
        test_pwrite = new[4];
        test_addr   = '{4, 4, 4, 4};
        test_pwrite = '{1, 1, 1, 0};

        env_o.agt.gen.count = 4;
        env_o.agt.gen.run(1, test_addr, test_pwrite);
    repeat(4 * 60) @(posedge vif.pclk);
        $display("[%0t] END OF SAME ADDR OVERWRITE TEST", $time);
        $finish;
    endtask
endclass
```

Tb Top

```
`timescale 1ns/1ps


module  tb_top #(parameter DATA_BUS_WIDTH=32,
parameter ADDR_BUS_WIDTH=32,
parameter TOTAL_TRANSACTIONS = 64
);
    import test_lib_pkg::*;

    bit pclk=0;
    bit presetn;
    base_test test;
    read_write rw;
    fixed_test ft;
```

```
    reset_test rt;
    mid_reset_test wt;
    overwrite_test ot;
    boundary_test bt;

    always #5 pclk=~pclk;

    apb_if #(.DATA_BUS_WIDTH(DATA_BUS_WIDTH),
.ADDR_BUS_WIDTH(ADDR_BUS_WIDTH)) vif(pclk);

    //DUT

    apb_v3_sram #(
            .ADDR_BUS_WIDTH(ADDR_BUS_WIDTH) ,
            .DATA_BUS_WIDTH(DATA_BUS_WIDTH),
        .MEMSIZE           (64),
        .MEM_BLOCK_SIZE      (8),
        .RESET_VAL           (0),
        .EN_WAIT_DELAY_FUNC  (1),
        .MIN_RAND_WAIT_CYC   (2),
        .MAX_RAND_WAIT_CYC   (2)
        ) DUT (.PCLK(vif.pclk), .PRESETn(vif.presetn),
           .PREADY(vif.pready), .PSLVERR(vif.pslverr), .PRDATA(vif.prdata),
                   .PSEL(vif.psel), .PENABLE(vif.penable),
.PWRITE(vif.pwrite),
                   .PADDR(vif.paddr), .PWDATA(vif.pwdata)
                    );

    //DETERMINE WHICH TEST TO RUN

    task run_test();

    if ($test$plusargs("RW")) begin
            $display("[%0t] RUNNING READ_WRITE TEST", $time);
            rw = new(vif);
            test = rw;

    end
    else if ($test$plusargs("FIXED")) begin
        $display("[%0t] RUNNING FIXED TEST", $time);
        ft=new(vif);
        test=ft;
    end
```

```
else if ($test$plusargs("RESET")) begin
    $display("[%0t] RUNNING RESET TEST", $time);
    rt=new(vif);
    test=rt;
end
else if ($test$plusargs("MID")) begin
    $display("[%0t] RUNNING MID TRANSACTION RESET CHECK TEST", $time);
    wt=new(vif);
    test=wt;
end
else if ($test$plusargs("OW")) begin
        $display("[%0t] RUNNING OVERWRITE TEST", $time);
        ot=new(vif);
        test=ot;
end
else if ($test$plusargs("BOUNDARY")) begin
        $display("[%0t] RUNNING BOUNDARY TEST", $time);
        bt=new(vif);
        test=bt;
end

else begin
        $display("[%0t] RUNNING BASE TEST", $time);
        test = new(vif);
end
endtask

initial begin
    $dumpfile("waveform.vcd");
    $dumpvars(1, tb_top);
end

initial begin


        run_test();
        test.total_transactions = TOTAL_TRANSACTIONS;

        fork
                test.env_o.agt.mon.run();
                test.env_o.agt.drv.run();
                test.env_o.sb.run();
        join_none
```

```
            test.run();

            wait(test.env_o.sb.compare_count == test.total_transactions);
            test.env_o.sb.report();
            $display("[%0t] ENV: WAIT CONDITION SATISFIED", $time);

            $finish;

    end

endmodule
```